



Sky: Diamond PAU

Security Review

Cantina Managed review by:

Christoph Michel, Lead Security Researcher

M4rio.eth, Lead Security Researcher

Xmxanuel, Lead Security Researcher

June 5, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Curve rate-limit deductions should round normalized amounts up	4
3.1.2	LayerZero <code>quoteTransfer</code> should not make a mutable proxy call to arbitrary targets	4
3.1.3	Uniswap V3 facet aggregated rate limits only work with pegged assets	5
3.1.4	Farm withdraw can claim rewards without the claim action key	5
3.1.5	Use safe <code>int256</code> casts	6
3.1.6	<code>rateLimit</code> key does not bind the actual token used in the ERC-20 transfer	6
3.1.7	Add slippage protection to Basin deposit and withdraw flows	7
3.1.8	<code>USDSFacet</code> 's allocator vaults should be configurable per Star	7
3.1.9	Controller <code>callSelector</code> s are not checked against hardcoded Controller selectors	8
3.2	Informational	8
3.2.1	<code>CCTPFacet</code> always uses standard, fee-free transfers	8
3.2.2	Rate limit increase can overflow before applying the cap	9
3.2.3	LayerZero <code>quoteTransfer</code> derives <code>minAmountLD</code> without <code>quote0FT</code> fee data	9
3.2.4	CCTP should reject a 100% fee cap	10
3.2.5	Some facets are still missing rate limits	10
3.2.6	Some withdraw paths do not restore paired deposit capacity	11
3.2.7	Uniswap V3 remove-liquidity slippage includes accrued fees	11
3.2.8	Apply aggregate deposit rate limit to actual deposited amounts in Curve	12
3.2.9	Implement slippage checks in facets instead of relying on integrations	13
3.2.10	<code>nonReentrant</code> applied inconsistently on functions without external calls	13
3.2.11	<code>removeIntegrations</code> is a "light" removal, facet specific storage persists	14
3.2.12	<code>MerklFacet.distributor</code> is now immutable (<code>v1.11</code> → <code>v1.12</code> migration change)	14
3.2.13	Align Basin rate-limit key hashing order with external parameter order	14
3.2.14	Harden and clarify controller shared storage addresses	15
3.2.15	Notes on <code>LIMIT</code> key values	15
3.2.16	<code>removeRelayer</code> emits removal event even if address was not a relayer	15
3.2.17	Minor issues, Typos & Documentation	16

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Apr 6th to Apr 26th the Cantina team conducted a review of [diamond-pau](#) on commit hash [fea256ad](#). The team identified a total of **26** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	9	7	2
Gas Optimizations	0	0	0
Informational	17	10	7
Total	26	17	9

The Cantina Managed team reviewed Sky's [diamond-pau](#) holistically on commit hash [a84fe961](#) and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Low Risk

3.1.1 Curve rate-limit deductions should round normalized amounts up

Severity: Low Risk

Context: CurveFacet.sol#L360, CurveFacet.sol#L384, CurveFacet.sol#L510, CurveFacet.sol#L540

Description: The CurveFacet rate limits use `_toNormalizedAmount(amount, rate)`, which floors:

```
normalizedAmount = amount * rate / 1e18;
```

For decimal-only Curve rates, this is usually exact. However, Curve `stored_rates()` can include non-power-of-ten rate-oracle or ERC4626 conversion rates. In those cases, flooring can under-deduct the aggregate rate limit by dust on each normalized conversion.

Rate-limit accounting should be conservative and overestimate the amount (rounding up). Repeated tiny malicious relayer (de-)allocations add up this value imprecision and ultimately lead to more value being transferred than defined by the rate limits.

Recommendation: Use a rounding-up normalized conversion for aggregate rate-limit deductions, e.g. `_toNormalizedAmountUp(amount, rate)`.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 LayerZero `quoteTransfer` should not make a mutable proxy call to arbitrary targets

Severity: Low Risk

Context: LayerZeroFacet.sol#L162, LayerZeroFacet.sol#L186

Description: `quoteTransfer` is public and accepts an arbitrary `oft` address. It then makes the ALM-Proxy perform a normal call into that address:

```
fee = abi.decode(
    IALMProxy(_getSharedControllerStorage().proxy).doCall(
        oft,
        abi.encodeCall(ILayerZeroLike.quoteSend, (sendParams, false))
    ),
    (MessagingFee)
);
```

This means anyone can make the ALMProxy call an arbitrary contract with the `quoteSend(SendParam,bool)` selector, without role checks and without a configured rate-limit key. The expected OFT implementation treats `quoteSend` as read-only, but PAU does not enforce that property. A malicious or mistaken target can execute state-changing logic under `msg.sender == ALMProxy` if it implements that selector.

It also makes off-chain quoting awkward because `quoteTransfer` is not `view`, so callers need to simulate a transaction instead of using a normal read.

Recommendation: Make `quoteTransfer` explicitly read-only by marking it `view` and using a static call through the proxy. The static context should propagate into `ALMProxy.doCall`, causing any state write in the target `quoteSend` to revert.

```
diff --git a/src/facets/layer-zero/ILayerZeroFacet.sol
    b/src/facets/layer-zero/ILayerZeroFacet.sol
@@
     function quoteTransfer(address oft, uint256 amount, uint32 destinationEndpointId)
         external
+         view
     returns (
         SendParam     memory sendParams,
```

```

        MessagingFee memory fee
    );
diff --git a/src/facets/layer-zero/LayerZeroFacet.sol
    ↪ b/src/facets/layer-zero/LayerZeroFacet.sol
@@
    function quoteTransfer(address oft, uint256 amount, uint32 destinationEndpointId)
        public
+       view
        override
        returns (SendParam memory sendParams, MessagingFee memory fee)
@@
-       fee = abi.decode(
-           IALMProxy(_getSharedControllerStorage().proxy).doCall(
-               oft,
-               abi.encodeCall(ILayerZeroLike.quoteSend, (sendParams, false))
-           ),
-           (MessagingFee)
-       );
+       (bool success, bytes memory returnData) =
    ↪ _getSharedControllerStorage().proxy.staticcall(
+       abi.encodeCall(
+           IALMProxy.doCall,
+           (oft, abi.encodeCall(ILayerZeroLike.quoteSend, (sendParams, false)))
+       )
+       );
+       require(success, "LayerZeroFacet/quoteSend-failed");
+       fee = abi.decode(abi.decode(returnData, (bytes)), (MessagingFee));
    }

```

Sky: Fixed in PR 199.

Cantina Managed: Fix verified.

3.1.3 Uniswap V3 facet aggregated rate limits only work with pegged assets

Severity: Low Risk

Context: [UniswapV3Facet.sol#L399](#)

Description: The current aggregate rate limit logic only works correctly for pegged asset pairs as it treats the token values as equal.

```

uint256 aggregateAmount =
    _toNormalizedAmount(token0, amounts.amount0) +
    _toNormalizedAmount(token1, amounts.amount1);

```

If the Uniswap V3 facet is used with non-pegged assets, this may produce incorrect behavior.

Recommendation: Clarify whether the Uniswap V3 facet will only be used with pegged assets. If non-pegged pairs are supported, replace the pegged-asset-specific logic.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.1.4 Farm withdraw can claim rewards without the claim action key

Severity: Low Risk

Description: `FarmFacet` exposes two ways to claim farm rewards:

```

claimReward(farm)
withdraw(farm, amount)

```

This is intentional at the interface level. `IFarmFacet.withdraw` says the function unstakes tokens and claims pending rewards.

The issue is that the two paths use different limit gates.

The standalone `claimReward` path requires the claim action key:

```
require(!_rateLimitExists(getClaimRewardRateLimitKey(farm)), "FarmFacet/invalid-action");
```

The `withdraw` path only consumes the withdraw key, then always calls `_claimReward(farm)` :

```
_decreaseRateLimit(getWithdrawRateLimitKey(farm), amount);

IALMProxy(_getSharedControllerStorage().proxy).doCall(
    farm,
    abi.encodeCall(IFarmLike.withdraw, (amount))
);

emit FarmWithdraw(farm, amount);

return _claimReward(farm);
```

`_claimReward` is the function that calls `getReward()` .

An Allocator can bypass the claim reward limit by calling `withdraw(1)` as long as there is a limit for withdraw.

Recommendation: Decide whether the withdraw is supposed to also authorize reward claims. We should also take into consideration that in case there is no reward limit, the withdraw should still succeed.

Sky: Fixed in PR 170.

Cantina Managed: Fix verified.

3.1.5 Use safe `int256` casts

Severity: Low Risk

Context: [CurveFacet.sol#L376](#)

Description: The rate limits are applied on `depositAmounts` and this parameter may be controlled by a compromised allocator, and casting these values to `int256` can overflow if a value exceeds `type(int256).max` . Although related arithmetic may already revert, relying on that is implicit and fragile.

Recommendation: Use explicit safe casts for all `uint256` to `int256` conversions, e.g. check `x <= uint256(type(int256).max)` before casting or use a safe casting helper.

Sky: Fixed in PR 167.

Cantina Managed: Fix verified.

3.1.6 `rateLimit` key does not bind the actual token used in the ERC-20 transfer

Severity: Low Risk

Context: [ERC4626Facet.sol#L85-L116](#), [AaveFacet.sol#L86-L119](#), [WEETHFacet.sol#L88-L178](#), [LayerZeroFacet.sol#L155-L232](#), [PendleFacet.sol#L114-L169](#), [CurveFacet.sol#L103-L204](#), [FarmFacet.sol#L40-L74](#), [ERC7540Facet.sol](#)

Description: Several facets currently construct rate-limit keys that do not bind the actual `token` and `to` address used for the ERC-20 transfers. A more secure rate-limit design would always include these values in the key, when they are not immutable in the facet itself.

For example in the generic `ERC4626Facet` , a malicious/hacked vault could return any asset address from `asset()` at deposit time. A rate limit given for `USDS` (18 decimals) could then be used for `USDC` (6 decimals) to steal a disproportionately large amount, or whichever ERC-20 in the ALMProxy has the highest per-unit value could be substituted as the `asset` .

Classification of `ApproveLib.approve` calls by whether `token` appears in the rate-limit key:

Token is in the rate-limit key:

- `BasinFacet` , `PSM3Facet` , `TransferAssetFacet` , `UniswapV3Facet` (swap and addLiquidity).

Token is NOT in the rate-limit key:

- `AaveFacet` (key on `aToken` , token is `aToken.UNDERLYING_ASSET_ADDRESS()`).
- `ERC4626Facet` , `ERC7540Facet` (key on `token` , actual asset is `token.asset()`).
- `LayerZeroFacet` (key on `(oft, eid)` , token is `oft.token()`).
- `PendleFacet` (key on `market` , token is `market.readTokens().pt`).
- `CurveFacet` (key on `pool` , tokens are `pool.coins(i)`).
- `FarmFacet` (key on `farm` , token is `farm.stakingToken()`).
- `WEETHFacet` (key is the bare `LIMIT_DEPOSIT` with no address; token is `weeth.eETH()`).

The attack requires the external contract to be upgradable by a third party or that the underlying token used for the ERC-20 transfers can be changed by governance.

Recommendation:: The worst-case loss of a protocol integration should be limited by its `rateLimits` and it should not be possible to exceed that amount.

Possible fixes: include the transferred token (and destination) directly in the rate-limit key so governance authorizes each token explicitly. Alternatively, cache the derived token as an `immutable` in single-integration facets like `WEETHFacet` . For the generic `ERC4626Facet` , store the `asset` (and `to` address) in facet-specific storage set at onboarding via a dedicated `setVaultAsset` admin function.

Sky: Fixed `rateLimit` keys in PR 148. `OTCBuffer._OTC_BUFFER_STORAGE_LOCATION` is fixed in version `v1.12.0-rc.1` .

Cantina Managed: Fix verified.

3.1.7 Add slippage protection to Basin deposit and withdraw flows

Severity: Low Risk

Context: `BasinFacet.sol#L58`

Description: `BasinFacet` deposit/withdraw operations currently do not enforce slippage bounds. Unlike other 4626-style flows in the codebase that use `minShares` or `maxSharePrice` , Basin users may receive fewer shares than expected on deposit or fewer assets than expected on withdrawal if pricing moves between quote and execution.

This is especially relevant because `Basin.totalAssets()` is less robust (for example, on redemption fulfillment) than alternatives like `PSM3`.

Recommendation: Consider adding user/admin-configurable slippage checks to Basin deposit and withdraw methods, such as `minShares` for deposits and `minAssetsOut` for withdrawals, or an equivalent max share price / min asset price bound.

Sky: Fixed in PR 138.

Cantina Managed: Fix verified.

3.1.8 `USDSFacet` 's allocator vaults should be configurable per Star

Severity: Low Risk

Context: `USDSFacet.sol#L44`

Description: The allocator `vault` address is specific to each Star, but it is currently defined as an `immutable` in the `USDSFacet` .

In a diamond/facet architecture, this would force deploying a separate facet for each Star instead of reusing a shared facet. Earlier controller deployments treated the vault as per-deployment configuration passed into `USDSLlib`, and that configurability was lost in this refactor.

Recommendation: Store the vault address in the `USDSFacet`'s storage rather than as an `immutable`, and add an admin setter so each Star can configure its own vault.

Sky: Fixed in commit `f0650f8`.

Cantina Managed: Fix verified.

3.1.9 Controller `callSelector` s are not checked against hardcoded Controller selectors

Severity: Low Risk

Context: `Controller.sol#L259`

Description: `Controller` does not call `_revertIfCallSelectorIsHardcoded(callSelector)` before dispatching through the fallback, relying on the assumption that the linked `Beacon` already enforces selector protection for this specific `Controller`'s public interface.

However, because the beacon address is not truly immutable, there is no guarantee that the controller and beacon will always agree on which selectors are hardcoded. A future facet could change the beacon address to one built for a different controller version, causing selector checks in the beacon to diverge from the controller's own routing assumptions.

Recommendation: Make the beacon address a real `immutable` in `Controller` rather than mutable shared storage. This should not interfere with the diamond structure as other facets should not need to read the Beacon address.

Sky: Fixed in commit `192f25a`.

Cantina Managed: Fix verified.

3.2 Informational

3.2.1 `CCTPFacet` always uses standard, fee-free transfers

Severity: Informational

Context: `CCTPFacet.sol#L76`

Description: CCTP v2 differentiates the standard and fast transfers by its `minFinalityThreshold` parameter, where `>= 2000` indicate standard transfers, less (usually value `1000` is used) indicates fast transfers that come with fees.

Standard Transfer attestation times The table below shows the average time for attestations to become available when using Standard Transfer (`minFinalityThreshold >= 2000`): <https://developers.circle.com/cctp/concepts/finality-and-block-confirmations#standard-transfer-attestation-times>

Option 2: Use Standard Transfer Change `minFinalityThreshold` to 2000 or higher to use Standard Transfer, which doesn't consume the Fast Transfer allowance. <https://developers.circle.com/cctp/concepts/fast-transfer-allowance#option-2-use-standard-transfer>

CCTP charges fees on Fast Transfers only. Standard Transfers are free. <https://developers.circle.com/cctp/concepts/fees>

Currently, the finality is hardcoded to `2000` (standard) for all transfers.

Recommendation: Clarify why the `DomainParameters` fee behavior exists if only fee-free standard transfers can be used. If fast transfers are supposed to be supported, consider adding another

`DomainParameters.allowFastTransfers` boolean parameter and let the relayer choose between standard and fast transfers (choosing `2000` and `1000`) if that boolean is set.

In addition, the `MAX_FINALITY_THRESHOLD` parameter should be called `MIN_FINALITY_THRESHOLD` .

Sky: Acknowledged. We renamed `MAX_FINALITY_THRESHOLD` parameter should be called `MIN_FINALITY_THRESHOLD` .

Cantina Managed: Acknowledged.

3.2.2 Rate limit increase can overflow before applying the cap

Severity: Informational

Context: `RateLimits.sol#L108`

Description: `triggerRateLimitIncrease` caps the new limit at `maxAmount` , but the addition happens before `_min` is applied:

```
d.lastAmount = newLimit = _min(currentRateLimit + amountToIncrease, maxAmount);
```

For a near- `type(uint256).max` configured limit, `currentRateLimit + amountToIncrease` can overflow and revert before `_min` can clamp the value to `maxAmount` . This is an edge case, but it means a route configured with very large finite limits can unexpectedly revert on a restorative action that should simply saturate at the cap.

Recommendation: Avoid performing the overflowing addition before the cap check. For example:

```
if (amountToIncrease >= maxAmount - currentRateLimit) {
    newLimit = maxAmount;
} else {
    newLimit = currentRateLimit + amountToIncrease;
}

d.lastAmount = newLimit;
```

Sky: Documented in PR 199:

```
// NOTE: This will revert if currentRateLimit + amountToIncrease overflows.
```

Cantina Managed: Fix verified.

3.2.3 LayerZero `quoteTransfer` derives `minAmountLD` without `quoteOFT` fee data

Severity: Informational

Context: `LayerZeroFacet.sol#L171`, `LayerZeroFacet.sol#L176`, `LayerZeroFacet.sol#L186`

Description: `quoteTransfer` computes `minAmountLD` by only rounding `amount` down to the OFT decimal conversion rate:

```
uint256 decimalConversionRate = ILayerZeroLike(oft).decimalConversionRate();
uint256 minAmountLD           = (amount / decimalConversionRate) * decimalConversionRate;
```

That value is then used both for quoting and for the actual transfer path:

```
sendParams = SendParam({
    dstEid      : destinationEndpointId,
    to          : recipient,
    amountLD    : amount,
    minAmountLD : minAmountLD,
    extraOptions : OptionsBuilder.newOptions().addExecutorLzReceiveOption(200_000, 0),
    composeMsg   : "",
    oftCmd      : ""
});
```

This assumes the OFT does not apply any fee or non-default debit/credit behavior. If an OFT charges a fee, the real `amountReceivedLD` can be lower than the decimal-normalized input, so `minAmountLD` can be too high and the transfer can revert. The previous implementation queried `quoteOFT` and used the returned receipt amount as the minimum amount.

Recommendation: Refactor `quoteTransfer` to do the same logical quote sequence as before, considering also if the other OFT has a fee enabled.

Sky: In order to get this over the line with minimal changes, we are going to keep `minAmountLD` as is and accept that this may not work with OFTs that have fees.

Cantina Managed: Acknowledged.

3.2.4 CCTP should reject a 100% fee cap

Severity: Informational

Context: CCTPFacet.sol#L121, CCTPFacet.sol#L166

Description: `setDomainParameters` accepts `maxFeeCapRate == _ONE_HUNDRED_PERCENT` :

```
require(maxFeeCapRate <= _ONE_HUNDRED_PERCENT, "CCTPFacet/max-fee-cap-rate-too-high");
```

During transfer, the actual per-message fee cap is derived as:

```
uint256 maxFee = (transferAmount * feeCapRate) / _ONE_HUNDRED_PERCENT;
```

If `feeCapRate == 10_000`, then `maxFee == transferAmount`. Circle's `TokenMessengerV2` rejects this case because `maxFee` must be strictly less than `amount` :

```
require(_maxFee < _amount, "Max fee must be less than amount");
```

So a configured 100% cap is accepted by PAU but cannot produce a successful CCTP burn.

Recommendation: Reject `maxFeeCapRate >= _ONE_HUNDRED_PERCENT` :

```
- require(maxFeeCapRate <= _ONE_HUNDRED_PERCENT, "CCTPFacet/max-fee-cap-rate-too-high");  
+ require(maxFeeCapRate < _ONE_HUNDRED_PERCENT, "CCTPFacet/max-fee-cap-rate-too-high");
```

Sky: Fixed in PR 199.

Cantina Managed: Fix verified.

3.2.5 Some facets are still missing rate limits

Severity: Informational

Context: DAIUSDSFacet.sol#L67, OTCFacet.sol#L174

Description: Rate limit existence checks have been added broadly across the codebase, but some facets are still missing them. This leaves these paths inconsistent with the rest of the facet/codebase and without the same pause or rate-limit control.

1. DAIUSDSFacet.sol#L67: `swapUSDStoDAI` and `swapDAIToUSDS` .
2. OTCFacet.sol#L174: `claim` has no rate limit but its `parameters.assetWhitelisted[assetToClaim]` implicitly acts as a whitelist / pausing functionality.

Recommendation: Consider adding `rateLimitExists` checks to the mentioned functions to align them with the rest of the codebase and ensure they can be paused if needed.

Sky: Fixed in PR 172, PR 175 and PR 194.

Cantina Managed: Fix verified.

3.2.6 Some withdraw paths do not restore paired deposit capacity

Severity: Informational

Context: [BasinFacet.sol#L106](#), [PSM3Facet.sol#L111](#), [FarmFacet.sol#L91](#)

Description: The expected rate-limit symmetry for simple deposit/withdraw adapters is:

```
deposit(asset, amount)
withdraw(asset, amount)
// net deposit capacity should be roughly unchanged
```

Some facets follow this model. For example, Aave withdraw decreases the withdraw key and restores the paired deposit key:

```
_decreaseRateLimit(getWithdrawRateLimitKey(aToken, pool), amountWithdrawn);
_tryIncreaseRateLimit(getDepositRateLimitKey(aToken, pool, underlying), amountWithdrawn);
```

ERC4626 does the same on both `withdraw` and `redeem`. USDS burn also restores the mint bucket.

Other deposit/withdraw adapters are one-way. They consume deposit capacity on entry, then only consume withdraw capacity on exit:

```
// Basin deposit
_decreaseRateLimit(getDepositRateLimitKey(basin, asset), amount);

// Basin withdraw
_decreaseRateLimit(getWithdrawRateLimitKey(basin, asset), assetsWithdrawn);
```

The same clean asymmetry exists in PSM3 and Farm:

- `PSM3Facet.deposit` consumes `getDepositRateLimitKey(asset)`, while `PSM3Facet.withdraw` only consumes `getWithdrawRateLimitKey(asset)`.
- `FarmFacet.deposit` consumes `getDepositRateLimitKey(farm, stakingToken)`, while `FarmFacet.withdraw` only consumes `getWithdrawRateLimitKey(farm)`.

So a deposit/withdraw round trip can leave the deposit bucket lower than before the round trip. The LP adapters also use independent add/remove budgets:

- Curve add-liquidity consumes aggregate and per-asset deposit/swap keys, while remove-liquidity only consumes the pool withdraw key.
- Uniswap V3/V4 add/increase consume aggregate and per-asset deposit keys, while remove/decrease only consumes the pool withdraw key.

That may be intentional for LP positions because fees, imbalance, token mix, and value accounting make the inverse less direct. For simple same-asset adapters like Basin and PSM3, the asymmetry is cleaner.

Recommendation: Decide whether withdraws should replenish paired deposit capacity or whether deposit capacity is intentionally one-way for these integrations.

Sky: Acknowledged. This is intended functionality. Rate limit recharging was added to AAVE, ERC4626 and PSM because these are high volume facets. 4626 can be utilization rate based integration, same with AAVE, where a rate is targeted and small amounts are frequently deposited and withdrawn. PSM was also used frequently in both directions.

Cantina Managed: Acknowledged.

3.2.7 Uniswap V3 remove-liquidity slippage includes accrued fees

Severity: Informational

Context: [UniswapV3Facet.sol#L453](#)

Description: The slippage check for a V3 liquidity decrease should apply to the actual amounts returned by `decreaseLiquidity`:

```
decreaseLiquidity(tokenId, liquidity, min) >= min
```

The facet already passes the mins into the position manager here:

```
amount0Min : min.amount0,  
amount1Min : min.amount1,
```

But after that, `removeLiquidity` calls `collect()` with `amount0Max` and `amount1Max` set to `type(uint128).max`, then computes `amounts` from the proxy balance delta:

```
amounts.amount0 = _getProxyBalance(token0) - startingToken0Balance;  
amounts.amount1 = _getProxyBalance(token1) - startingToken1Balance;
```

Those deltas can include withdrawn principal plus accrued fees. The second PAU-side check then does:

```
_checkSlippage(amounts.amount0, min.amount0, maxSlippage);  
_checkSlippage(amounts.amount1, min.amount1, maxSlippage);
```

and `_checkSlippage` requires `minAmount >= amount * maxSlippage / 1e18`. So if accrued fees make `amounts.amount0` or `amounts.amount1` larger, the user-provided min must also cover those fees.

V4 does not have this extra post-balance slippage check. It passes the mins into the `DECREASE_LIQUIDITY` action and uses the returned amounts for accounting.

Recommendation: Align the behavior with v4 implementation.

Sky: Fixed in [PR 176](#).

Cantina Managed: Fixed by collecting fees before computing the balance delta. This leads to the same behavior as the previous UniswapV3 library.

3.2.8 Apply aggregate deposit rate limit to actual deposited amounts in Curve

Severity: Informational

Context: [CurveFacet.sol#L185](#)

Description: The desired invariant is that the rate limits of the following actuals should be the same:

```
addLiquidity(depositAmounts) ≈ swap(swappedInAmounts) + addLiquidity(resultingUnderlying)
```

The aggregate deposit rate limit is currently applied to the initial `depositAmounts` value before `addLiquidity`. For unbalanced deposits, this can overstate the actual deposited value because implicit swaps and imbalance fees reduce the resulting underlying/claimable amounts.

The aggregate value should be taken on the post-`addLiquidity` deposited amounts.

Recommendation: Instead of rate limiting the aggregate value *at the start*, consider doing it in `_decreaseRateLimitsForAddLiquidity` on the *actual* deposited/claimable values, for example:

```
// pseudo code  
// _decreaseRateLimitsForAddLiquidity  
for (uint256 i = 0; i < tokens.length; ++i) {  
    address token = tokens[i];  
  
    int256 swappedIn = swappedInAmounts[i];  
    int256 deposited = int256(depositAmounts[i]) - swappedIn;  
    if (deposited > 0) {  
+       valueDeposited += uint256(deposited) * rates[i];  
        _decreaseRateLimit(getAssetDepositRateLimitKey(pool, token),  
            ↳ uint256(deposited));  
    }  
  
    if (swappedIn > 0) {  
        _decreaseRateLimit(getSwapRateLimitKey(pool, token), uint256(swappedIn));  
    }  
}
```

```

}

+ valueDeposited /= 1e18;
+ _decreaseRateLimit(getAggregateDepositRateLimitKey(pool), valueDeposited);

```

Note that the current initial `valueDeposited` can still be used for the `minLpAmount >= valueDeposited * maxSlippage / ICurvePoolLike(pool).get_virtual_price()` slippage check. Consider renaming `valueDeposited` to something like `valueInput` as it doesn't represent the *deposited* value in this case.

Sky: Fixed in PR 192.

Cantina Managed: Fix verified.

3.2.9 Implement slippage checks in facets instead of relying on integrations

Severity: Informational

Context: `CurveFacet.sol#L151`, `CurveFacet.sol#L210`, `UniswapV4Facet.sol#L484`

Description: The code was changed to avoid trusting return values from integrated protocols by computing output amounts via balance deltas. However, it still trusts those protocols to perform slippage checks against `minAmountOut`, which is inconsistent and makes the trust model unclear.

Recommendation: After computing `amountOut` from pre/post-balance deltas, explicitly implement the slippage checks in the facet, or clarify why integration's internal slippage enforcement is trusted but their return values are not.

Sky: Fixed in PR 191 and PR 192.

Cantina Managed: Fix verified.

3.2.10 `nonReentrant` applied inconsistently on functions without external calls

Severity: Informational

Context: `Beacon.sol`, `Controller.sol`, `AccessControls.sol`, `RateLimits.sol`, `ALMProxy.sol`, `ALMProxyFreezable.sol`

Description: `nonReentrant` is applied inconsistently across the state-modifying externals of the contracts.

Classification by whether they make external 3rd-party calls.

Has `nonReentrant`, makes no 3rd-party call (defensive guard only):

- `Beacon.setIntegration`, `Beacon.removeIntegration`.
- `Controller.updateIntegrations`, `Controller.removeIntegrations` (only external call is `IBeacon.getConfigs`, a trusted `view` read).
- `AccessControls.removeRelayer`.

No `nonReentrant`, makes no 3rd-party call (inconsistent with the group above):

- `RateLimits.setRateLimitData` (all variants), `setUnlimitedRateLimitData`.
- `RateLimits.triggerRateLimitDecrease`, `triggerRateLimitIncrease`.
- `ALMProxyFreezable.removeRelayer`.

No `nonReentrant`, DOES make 3rd-party call (the group where reentrancy matters most):

- `Controller.fallback`: `delegatecall` into facet (intentional; each facet owns its own `nonReentrant`).
- `ALMProxy.doCall`, `doCallWithValue`, `doDelegateCall`: call to any target.
- `ALMProxyFreezable.doCall`, `doCallWithValue`: call to any target.

None of these paths is exploitable today. The proxy `do*` functions are protected by role-gating plus the facet-level guard via the Controller.

The issue is the small **inconsistency**, some callback-free admin paths carry `nonReentrant`, others don't.

Recommendation: Pick one convention and apply uniformly.

Sky: Acknowledged, will leave as is.

Cantina Managed: Acknowledged.

3.2.11 `removeIntegrations` is a "light" removal, facet specific storage persists

Severity: Informational

Context: `Controller.sol#L108`

Description: `removeIntegrations` clears only the Controller's `configs` and `dispatches` for the given IDs. It does not touch:

1. **Facet ERC-7201 storage** for example `AaveFacet.maxSlippages`, `UniswapV4Facet.tickLimits`. If the same facet is re-onboarded later, the old per-token/per-pool state is still live and immediately in effect.
2. **Rate-limit entries** in `RateLimits` persist across integration add/remove.
3. **Token approvals** that the proxy granted to protocol contracts during past operations. A review of current facets shows each one either consumes the approval in-call (Aave, ERC4626, Curve, etc...) or resets it to zero (UniswapV4). But this is a facet-by-facet invariant, not enforced architecturally.

Recommendation: Document the "light removal" semantics clearly.

Sky: Acknowledged. This is accepted and intentional: operators who want a clean removal should first zero-out facet admin state via the facet's own setters, then call `removeIntegrations`.

Cantina Managed: Resolved.

3.2.12 `MerklFacet.distributor` is now immutable (`v1.11` → `v1.12` migration change)

Severity: Informational

Context: `MerklFacet.sol#L28-L38`, `MainnetController.sol#L224`

Description: Change from `v1.11`: `distributor` is now an `immutable` set in the constructor (with a zero-address check), instead of a storage variable mutable via `setMerklDistributor`.

Operational implication: supporting multiple Merkl distributors now requires deploying a separate `MerklFacet` per distributor.

Recommendation: Informational only, expected migration behavior. No action needed.

Sky: Fixed in PR 129.

Cantina Managed: Fix verified.

3.2.13 Align Basin rate-limit key hashing order with external parameter order

Severity: Informational

Context: `BasinFacet.sol#L114`

Description: The hash input order currently places `asset` before `basin`, which is inconsistent with the external function signatures that take `basin` before `asset`. This makes the encoding less intuitive.

Recommendation: Update the hashing/encoding order to place `basin` before `asset` so it matches the external function parameter order, unless there is a documented compatibility reason to preserve the current ordering.

Sky: Acknowledged. The convention we've been using so far is key, asset, destination which we want to keep.

Cantina Managed: Acknowledged.

3.2.14 Harden and clarify controller shared storage addresses

Severity: Informational

Context: `IController.sol#L71`

Description: `proxy`, `rateLimits`, and `accessControls` are described as "immutable", but they are currently stored in shared storage so facets can access them during `delegatecall`. This naming is misleading because they are not Solidity immutables and can still be modified by facet logic, intentionally or accidentally.

Recommendation: Update the documentation to reflect that they are shared controller configuration stored in storage rather than Solidity-level immutables. Clarify the intended mutability guarantees and evaluate a hardening approach that preserves facet accessibility while preventing or detecting unintended changes.

Sky: Fixed in commits `b00c188` and `40a96db`.

Cantina Managed: Fix verified.

3.2.15 Notes on `LIMIT` key values

Severity: Informational

Context: `CentrifugeFacet.sol#L79-L81`

Description: While preventing collisions for facet functions is dynamically solvable at runtime now due to the `callSelector s → (facet, delegateSelector)` mappings, checking for collisions in `LIMIT` keccak key values is not done automatically and must currently be manually reviewed and enforced.

The `CentrifugeFacet` currently defines and uses keys (and therefore rate limit logic) from another facet (`ERC7540Facet`). While this is intentional in this case, it might not always be obvious and dependence on another `facet`'s behavior is fragile code as facets can be updated independently.

Recommendation: Consider coming up with a convention for the `LIMIT` key values for a facet, like each facet prefixing the keys with their name, as in `LIMIT_CENTRIFUGE_` here, otherwise, a comment justifying using a different facet's key should be required.

In addition, research ways to automatically check for `LIMIT` key value collisions.

Alternatively, consider making `CentrifugeFacet` a standalone facet, right now, it's a cancelation add-on to the existing `ERC7540` facet and can't be used by itself.

Sky: Acknowledged. We've spoken internally and for integrations like Maple, Ethena and Centrifuge that extend standards we'd rather just use the standard facet and then a dedicated facet for the extension (current state).

Cantina Managed: Acknowledged.

3.2.16 `removeRelayer` emits removal event even if address was not a relayer

Severity: Informational

Context: `ALMProxyFreezable.sol#L35`, `AccessControls.sol#L36`

Description: The `removeRelayer(relayer)` function in `ALMProxyFreezable` and `AccessControls` does not check the success return value of the internal `_revokeRole` call and always emits the `RelayerRemoved(relayer)` event.

If the provided address is not a relayer, the event can be misleading.

Recommendation: Consider checking the returning value and reverting if the relayer was not removed, or only emitting the event if it was removed.

Sky: Fixed in commit `e1d0a51`.

Cantina Managed: Fix verified.

3.2.17 Minor issues, Typos & Documentation

Severity: Informational

Context: (See each case below)

Description:

- `IAccessControls.sol#L11`: "freezer" role is a bit ambiguous because we don't necessarily have a freezer role now, we have an admin of the allocator that can freeze.
- `IRateLimits.sol#L131`: Typo, should be `* @notice Triggers the rate limit for a specific key and reduces the available amount by the provided value`.
- `IRateLimits.sol#L156`: The comment should be `/// @notice Role identifier for controller accounts that can increase/decrease limits`.
- `IFarmFacet.sol#L26`: Add `amount` parameter.
- `PSMFacet.sol#L194`: Should be added the DAI as well: `// NOTE: As swaps are only done between USDC, DAI and USDS, no need for ApproveLib`.
- `ICCTPFacet.sol#L80`: Comment should be `/// @notice Always bytes32(0): any address can complete the message on the destination chain`.
- `AccessControls.sol#L15`: The reentrancy guard can be removed, no function is using it.
- `ILayerZeroFacet.sol#L82`: The interface still says excess native fee is refunded to the caller, but rc2 sweeps controller-held excess ETH to the `ALMProxy`.
- `README.md#L97`: README still says `FREEZER_ROLE` can stop compromised allocators via `removeAllocator`, but the live `AccessControls` path is role-admin based. `removeAllocator` only exists on `ALMProxyFreezable`, not the deployed `ALMProxy` path.

Recommendation: Consider fixing the naming, links, and suggestions mentioned above.

Sky: Fixed in PR 167.

Cantina Managed: Verified.